AutoDocLM

Repository docs from static analysis and hierarchical LLM inference

Problem insight

Three failure modes of naive documentation

1

Context overflow

A medium-sized project with 100-200 source files generates more tokens than any model can process in one pass. The approach fails before it begins.

2

Hallucinated architecture

Without grounding evidence, the model invents plausible but wrong descriptions of cross-module relationships, data flows, and dependency chains.

3

Irreversible signal loss

Cross-file function calls vanish as summaries compress through each stage. The most architecturally significant information is lost first and can never be recovered.

Deterministic artifacts travel with every inference. No exceptions.

Raw code enters the LLM once. After that, only structured JSON and static analysis.

01 Deterministic preprocessing

Clone repo, filter files, AST-chunk source code with decorator and parent-class context, compute all six static analysis artifacts. No LLM.

02 Hierarchical LLM inference

Four levels: chunk and file summaries via DeepSeek-Coder-6.7B, folder and repo architecture via Qwen2.5-7B. Each prompt receives prior summaries plus graph artifacts.

03 Documentation writing

Qwen2.5-7B converts structured JSON summaries into Markdown pages. "Unknown" propagation enforced here. Writing prompts consume JSON, not prose.

04 Assembly

Mermaid diagrams generated deterministically from networkx graph data. MkDocs builds the final static documentation site.

Six deterministic artifacts
Ground every inference with facts

file_manifest.json
Filtered inventory of all valid files. Test files are tagged but retained for completeness.

import_graph.json
Directed module dependency graph with alias normalization. "np" resolves to "numpy" so references match edges.

cross_file_calls.json
Maps which functions in one file call which functions in another. Preserves connections summarization destroys.

centrality_scores.json
Per-file betweenness centrality from the import graph. Prioritizes which summaries get full context space.

folder_graphs
Per-folder dependency summaries showing incoming, outgoing, and internal edges. Grounds folder-level inference.

entrypoints.json
Detected execution entrypoints with confidence scores. If none are found, the list stays empty. No fabrication.

Deep dive

Preserves what summarization destroys

Schema: calling file → function → outbound calls

```
order_service.py
→ process_order
calls user_service.py · get_user
calls db.py · commit
```

Injected into every file-level summary prompt at Step 7.2, giving the model precise caller → callee resolution across files.

The concrete difference

"The API layer depends on the service layer."

Noise. No function-level detail survives summarization.

"The API layer calls get_user in the service layer to resolve user identity before processing orders."

Actionable architecture. Precise cross-file function resolution.

Without this artifact, the model knows imports but not which functions call which. The call graph closes that gap deterministically.

01 Chunk

DeepSeek-Coder 6.7B reads raw source code. Outputs structured JSON capturing purpose, inputs, outputs, side effects, and calls.

02 File

DeepSeek-Coder 6.7B merges chunk summaries with import graph and cross-file calls. Outputs file-level role, symbols, and interactions.

03 Folder

Qwen2.5 7B ranks file summaries by centrality. Top 5 in full, rest abbreviated. Adds folder graph edges and centrality scores.

04 Repository

Qwen2.5 7B reads all folder summaries priority-ordered. Outputs repo_architecture.json covering purpose, architecture, and data flow.

Reliability

Four mechanisms enforce reliability

"Unknown" propagation

If any source field contains "unknown," output must state "Not confirmed in repository." Fails loudly, never silently.

JSON validation + retry

Every response is validated via json.loads(). On failure, one retry with a stricter prompt. On second failure, error marker. Pipeline never crashes.

Fixed seed + low temp

Temperature 0.1, fixed seed per run, top_p 0.9. Same repo run twice produces effectively identical summaries.

Content-aware caching

Cache keys use sha256(content + model + prompt version). Prompt changes invalidate all dependent stages. Reruns take minutes, not hours.

Each mechanism targets a different failure class. Together they guarantee the pipeline fails loudly, retries gracefully, and reproduces exactly.

A complete documentation website, generated end to end

The pipeline writes a full file tree under outputs/repo_name/: index, setup, architecture, per-module and per-file pages, plus a built MkDocs static site. Two Mermaid diagrams ship with every run. The folder-level component graph stays readable at any repo size. The file-level import graph filters to the top 30 files by centrality to prevent visual overload.

Narrative quality comes from the two-model split. DeepSeek handles code-level precision at chunk and file stages. Qwen handles long-form technical prose at folder, repo, and writing stages.
The result: Markdown pages that describe real interactions grounded in graph evidence, not vague architectural hand-waving.

Scale
Constraints and prioritization

SMALL	1K–10K source lines 10–80 files, 50–500 estimated chunks	10K
MEDIUM	10K–30K source lines 80–200 files, 500–1,500 estimated chunks	30K
HARD LIMIT	Processing ceiling 300 files maximum, 2,000 chunks maximum	300
OVERFLOW	Centrality ranking High-betweenness files first, vendor paths skipped	Top

Three conclusions

Deterministic artifacts are first-class citizens. The import graph, cross-file call graph, and folder graphs travel alongside LLM summaries at every inference stage. This prevents hallucinated architecture.

Source code enters the model exactly once. Every subsequent stage operates on structured JSON and graph evidence. This makes the pipeline tractable at any supported scale.

Uncertainty is propagated, never suppressed. When the system lacks knowledge, the documentation says so. Documentation that admits gaps is more trustworthy than documentation that fills them with plausible fiction.

Repository:
https://github.com/KaushalrajPuwar/AutoDocLM